End-to-End ML on Snowflake

Git Integration, Model Promotion & Production Scheduling

From Notebooks to Production Pipelines — A Hands-On Walkthrough

Agenda

#	Topic	What You Will Do
1	Architecture Overview	Understand the end-to-end ML lifecycle on Snowflake
2	Project Structure	See how notebooks evolve into production code
3	Snowflake ML Platform	Feature Store, Model Registry, SPCS, Online Serving
4	Git Integration Setup	Connect your GitHub/GitLab repo to Snowflake
5	Team Collaboration	Pull, branch, develop, and push from Snowflake
6	Model Promotion	Promote notebook code to production `.py` modules
7	Scheduled Execution	Run pipelines from Git via Snowflake Tasks
8	Hands-On Implementation	Build it yourself in your environment

The Problem We Are Solving

Without This Pattern

- Notebooks live on laptops — no version control
- Copy-paste code between environments
- Manual model deployment (export `.joblib`, upload, pray)
- No audit trail for model versions
- Scheduling = a cron job on someone's laptop
- Team steps on each other's work

With This Pattern

- All code in Git — full version history
- Snowflake fetches code directly from Git
- Models registered in Snowflake Model Registry
- `EXECUTE IMMEDIATE FROM` runs `.py` from Git
- Snowflake Tasks schedule recurring pipelines
- Branching + PRs for safe team collaboration

Architecture Overview

```
LOCAL DEVELOPMENT                    GITHUB                          SNOWFLAKE

┌──────────────────┐   git push    ┌──────────────┐   FETCH    ┌──────────────────────────────┐
│ notebooks/       │ ──────────────>│ main branch  │ ──────────>│ GIT REPOSITORY object        │
│ src/             │               │              │            │ @.../branches/main/          │
│ production/      │               │ feature/*    │            │                              │
│                  │<──────────────│ tags/        │            │ EXECUTE IMMEDIATE FROM       │
│ Experiment in    │   git pull    │              │            │                              │
│ notebooks        │               │              │            │ ┌──────────┐  ┌──────────┐  │
│ Promote to src/  │               │              │            │ │ TASK:    │─>│ TASK:    │  │
│ Test locally     │               │              │            │ │ GIT_FETCH│  │ BATCH_   │  │
└──────────────────┘               └──────────────┘            │ └──────────┘  │ SCORING  │  │
                                          │ PR + Review       │               └──────────┘  │
                                          v                   │                             │
                                   ┌──────────────┐           │ MODEL REGISTRY              │
                                   │ Merge to     │           │ FEATURE STORE               │
                                   │ main         │──── tag ──>│ BATCH_PREDICTIONS table     │
                                   └──────────────┘           └──────────────────────────────┘
```

Project Structure — From Notebooks to Production

Repository Layout

```
healthcare-readmission-ml/
├── notebooks/          <-- Experimentation
│   ├── 01_local_training.ipynb
│   ├── 02_snowflake_setup.ipynb
│   ├── 03_model_registry.ipynb
│   ├── 04_batch_inference.ipynb
│   └── 05_realtime_inference.ipynb
├── src/                <-- Production modules
│   ├── config.py
│   ├── feature_engineering.py
│   ├── train.py
│   ├── register_model.py
│   ├── batch_inference.py
│   └── realtime_inference.py
├── production/         <-- Snowflake entry points
│   ├── run_training.py
│   ├── run_batch_inference.py
│   └── tasks/setup_tasks.sql
├── scripts/            <-- Git + promotion
│   ├── setup_snowflake_git.sql
│   └── promote_to_prod.sh
└── artifacts/
    └── model_metadata.json
```

The Three Layers

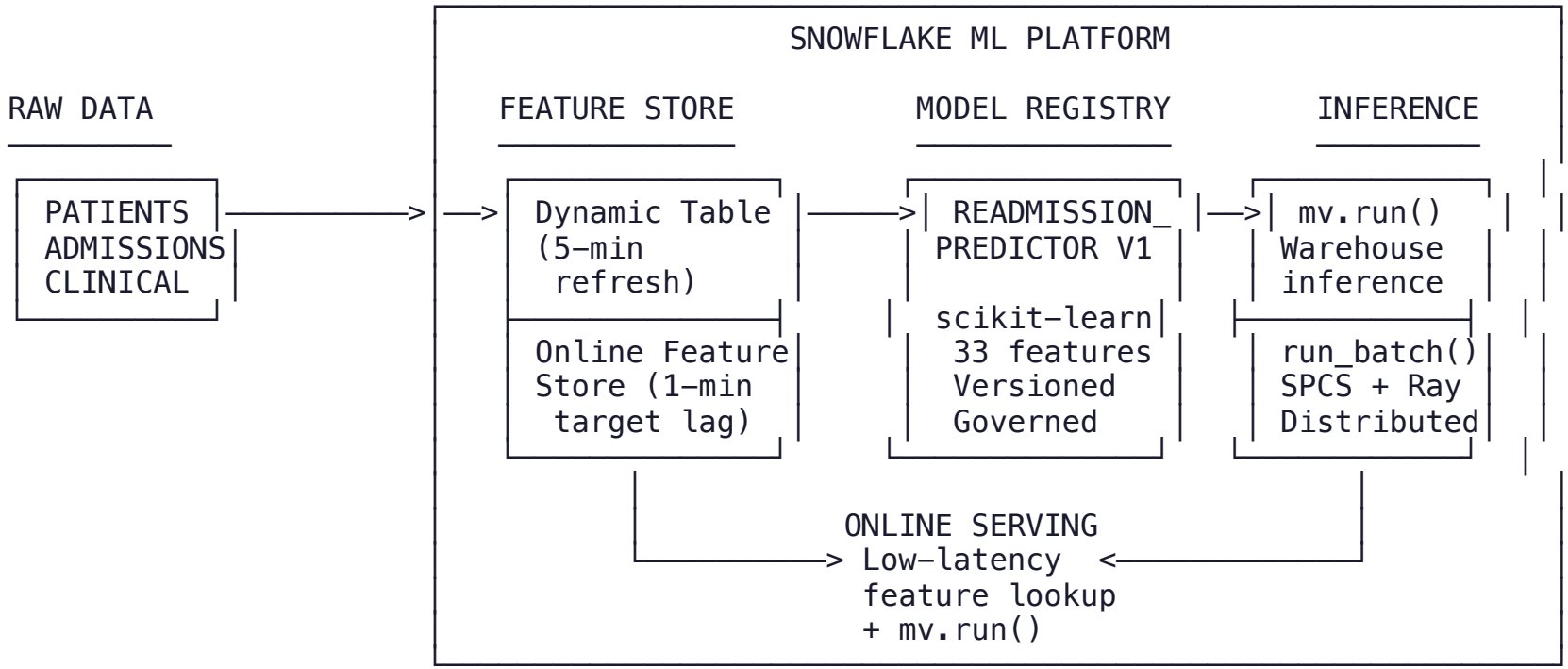
Layer	Purpose	Who Uses It
`notebooks/`	Experimentation, EDA, prototyping	Data Scientists
`src/`	Reusable, tested Python modules	DS + ML Engineers
`production/`	Entry points for Snowflake execution	Snowflake Tasks / Ops

Key Principle

> *Notebooks are for **discovery**.*
> `src/` *is for **reusable logic**.*
> `production/` *is for **execution in Snowflake**.*

Code flows **one direction**: notebooks --> src --> production.

Snowflake ML Platform Features Used



Feature Store Deep Dive

What It Provides

Capability	Implementation
Entity	`PATIENT` (join key: `PATIENT_ID`)
Feature View	`PATIENT_CLINICAL_FEATURES` V1
Backing	Dynamic Table, 5-min refresh
Online Store	1-min target lag for real-time
Features	33 engineered columns
Lineage	Raw tables --> Features --> Training Dataset

Feature Categories (33 total)

Category	Count	Examples
Demographics	4	`AGE`, `GENDER_ENC`, `INSURANCE_ENC`
Admission	6	`LENGTH_OF_STAY`,

Feature View SQL (excerpt)

```sql
SELECT
  a.PATIENT_ID,
  a.ADMISSION_ID,
  TO_TIMESTAMP(a.DISCHARGE_DATE) AS EVENT_TIMESTAMP,
  -- Demographics
  p.AGE,
  CASE WHEN p.GENDER = 'M' THEN 1 ELSE 0 END
    AS GENDER_ENC,
  -- Clinical flags
  CASE WHEN c.BNP > 500 THEN 1 ELSE 0 END
    AS HIGH_BNP,
  -- Historical (window functions)
  COUNT(*) OVER (
    PARTITION BY a.PATIENT_ID
    ORDER BY a.ADMIT_DATE
    ROWS BETWEEN UNBOUNDED PRECEDING
      AND 1 PRECEDING
  ) AS PRIOR_ADMISSIONS_6M
FROM RAW_DATA.ADMISSIONS a
JOIN RAW_DATA.PATIENTS p ...
JOIN RAW_DATA.CLINICAL_MEASUREMENTS c ...
```

> *The same SQL drives both the Dynamic Table (automated refresh) and the Online Feature Store*

Model Registry & Inference

Model Registration

```python
from snowflake.ml.registry import Registry

registry = Registry(
    session=session,
    database_name="HEALTHCARE_ML",
    schema_name="MODEL_REGISTRY"
)

mv = registry.log_model(
    model=model,                        # sklearn model
    model_name="READMISSION_PREDICTOR",
    version_name="V1",
    sample_input_data=sample_input, # 100 rows
    conda_dependencies=["scikit-learn"],
    metrics={
        "roc_auc": 0.552,
        "average_precision": 0.2895,
        "n_features": 33
    },
    task=ml_task.Task
        .TABULAR_BINARY_CLASSIFICATION
)
```

Three Inference Patterns

Pattern	Method	Best For
Warehouse	`mv.run(df)`	Small-medium datasets, ad-hoc
SPCS Batch	`mv.run_batch()`	Millions of rows, distributed via Ray
Real-Time	Online Feature Store + `mv.run()`	Single-patient, low-latency

Batch Inference on SPCS

```python
job = mv.run_batch(
    compute_pool="DEMO_POOL_CPU",
    X=features_df,
    output_spec=OutputSpec(
        stage_location="@.../BATCH_OUTPUT/",
        mode=SaveMode.OVERWRITE
    ),
    job_spec=JobSpec(
        function_name="predict_proba"
```

Part 2: Git Integration & Team Workflow

Connecting your repository to Snowflake and running code directly from Git

Git Integration — How It Works

```
GITHUB / GITLAB                                SNOWFLAKE

                      API Integration
                      (HTTPS access)
  Your Repo                                   GIT REPOSITORY
                  <------------------------>   (read-only clone)
  main branch         ALTER ... FETCH
  feature/*                                   Accessible as a
  tags/                                       Snowflake STAGE:
                                              @DB.SCHEMA.REPO/
                                               branches/main/
                                               branches/feature/
      |                                        tags/v1.0/
      |  push
      v                                              |
                                                     v
  Data                                        EXECUTE IMMEDIATE
  Scientist                                   FROM @.../main/
  Laptop                                        production/
                                                run_*.py
```

Git Integration — Setup (Hands-On)

Three Objects to Create

Step 1: API Integration (account-level, one-time)

```
CREATE OR REPLACE API INTEGRATION
  GITHUB_API_INTEGRATION
  API_PROVIDER = git_https_api
  API_ALLOWED_PREFIXES = (
    'https://github.com/YOUR-ORG/'
  )
  ALLOWED_AUTHENTICATION_SECRETS = ALL
  ENABLED = TRUE;
```

Step 2: Secret (for private repos)

```
CREATE OR REPLACE SECRET
  HEALTHCARE_ML.GIT_INTEGRATION.GITHUB_SECRET
  TYPE = password
  USERNAME = 'your-github-user'
  PASSWORD = '<GITHUB_PAT>';
```

> *For **public** repos, the secret is optional.*

Step 3: Git Repository Object

```
CREATE OR REPLACE GIT REPOSITORY
  HEALTHCARE_ML.GIT_INTEGRATION
    .HEALTHCARE_ML_REPO
  ORIGIN = 'https://github.com/YOUR-ORG/
    healthcare-readmission-ml.git'
  API_INTEGRATION = GITHUB_API_INTEGRATION
  GIT_CREDENTIALS =
    HEALTHCARE_ML.GIT_INTEGRATION.GITHUB_SECRET
  ;
```

Step 4: Fetch & Verify

```
ALTER GIT REPOSITORY
  HEALTHCARE_ML.GIT_INTEGRATION
    .HEALTHCARE_ML_REPO FETCH;

-- Browse the repo from SQL
LIST @HEALTHCARE_ML.GIT_INTEGRATION
  .HEALTHCARE_ML_REPO/branches/main/;
```

Browsing & Executing Code from Snowflake

Browse Files via SQL

```sql
-- Top-level directory
LIST @...HEALTHCARE_ML_REPO
  /branches/main/;

-- Production scripts
LIST @...HEALTHCARE_ML_REPO
  /branches/main/production/;

-- Feature branch code
LIST @...HEALTHCARE_ML_REPO
  /branches/feature/improve-bnp/;

-- Show branches
SHOW GIT BRANCHES IN
  HEALTHCARE_ML.GIT_INTEGRATION
    .HEALTHCARE_ML_REPO;
```

Read File Contents

```sql
SELECT $1 FROM
  @...HEALTHCARE_ML_REPO
  /branches/main/src/config.py;
```

Execute Python from Git

```sql
-- Run batch inference
EXECUTE IMMEDIATE FROM
  @HEALTHCARE_ML.GIT_INTEGRATION
    .HEALTHCARE_ML_REPO
    /branches/main/production
    /run_batch_inference.py;

-- Train + register a new version
EXECUTE IMMEDIATE FROM
  @HEALTHCARE_ML.GIT_INTEGRATION
    .HEALTHCARE_ML_REPO
    /branches/main/production
    /run_training.py
  USING (VERSION => 'V2');
```

> `EXECUTE IMMEDIATE FROM` *runs the Python file directly inside Snowflake's execution environment. No file copying, no uploads. It reads from the Git stage.*

Key Benefit

Team Access Control

Role-Based Access to Git and ML Objects

```sql
CREATE ROLE IF NOT EXISTS ML_ENGINEER;

-- Git repo: read-only (fetch + execute, not push)
GRANT READ ON GIT REPOSITORY
  HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO
  TO ROLE ML_ENGINEER;

-- ML schemas: appropriate access per schema
GRANT SELECT ON ALL TABLES IN SCHEMA HEALTHCARE_ML.RAW_DATA      TO ROLE ML_ENGINEER;
GRANT CREATE MODEL ON SCHEMA HEALTHCARE_ML.MODEL_REGISTRY        TO ROLE ML_ENGINEER;
GRANT CREATE TABLE ON SCHEMA HEALTHCARE_ML.INFERENCE              TO ROLE ML_ENGINEER;
```

Role	Git Access	Raw Data	Feature Store	Model Registry	Inference
`ACCOUNTADMIN`	Full	Full	Full	Full	Full
`ML_ENGINEER`	Read + Execute	SELECT	SELECT	CREATE MODEL	CREATE TABLE/STAGE
`ANALYST`	Read	SELECT	SELECT	--	SELECT

> ***No ACCOUNTADMIN in production.*** *The* `ML_ENGINEER` *role has exactly the permissions needed.*

Part 3: Model Promotion & Scheduling

Moving from notebooks to production, and automating execution

The Promotion Flow

```
EXPERIMENTATION                    PROMOTION                    PRODUCTION
---------------                    ---------                    ----------

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────────────┐
│                  │   │                  │   │                          │
│ NOTEBOOKS        │   │ src/ MODULES     │   │ SNOWFLAKE EXECUTION      │
│                  │   │                  │   │                          │
│ 01_local_        │   │ config.py        │   │ EXECUTE IMMEDIATE        │
│   training.ipynb │   │ train.py         │   │   FROM @git_repo/        │
│ 02_snowflake_    │   │ register_model   │   │   .../run_training.py    │
│   setup.ipynb    │   │   .py            │   │                          │
│ 03_model_        │   │ batch_inference  │   │ Scheduled via TASK       │
│   registry.ipynb │   │   .py            │   │ - GIT_FETCH_TASK         │
│ 04_batch_        │   │ realtime_        │   │ - BATCH_SCORING_TASK     │
│   inference      │   │   inference.py   │   │                          │
│ 05_realtime_     │   │ feature_         │   │ Tagged releases:         │
│   inference      │   │   engineering    │   │   model-V1               │
│                  │   │   .py            │   │   model-V2               │
└──────────────────┘   └──────────────────┘   └──────────────────────────┘
 Extract ──────>        promote ──────>

                          │
                          │ git push + PR
                          v
                   ┌──────────────────┐
                   │ Code Review      │
                   │ Merge to main    │
                   └──────────────────┘
```

Notebook to Module — What Changes

Notebook (experimentation)

```python
# 03_model_registry.ipynb — cell 3
session = Session.builder.config(
    "connection_name", "DEMO"
).create()
session.sql("USE ROLE ACCOUNTADMIN").collect()
session.sql("USE DATABASE HEALTHCARE_ML").collect()
session.sql("USE SCHEMA MODEL_REGISTRY").collect()
session.sql(
    "USE WAREHOUSE HEALTHCARE_ML_WH"
).collect()
```

- Hardcoded connection, role, warehouse
- Inline SQL mixed with Python
- No reusability across notebooks
- No environment switching

Production Module (src/)

```python
# src/config.py
ENV = os.environ.get(
    "HEALTHCARE_ML_ENV", "DEV"
).upper()

_ENVIRONMENTS = {
    "DEV": {
        "connection_name": "DEMO",
        "role": "ACCOUNTADMIN",
        "warehouse": "HEALTHCARE_ML_WH",
        ...
    },
    "PROD": {
        "connection_name": "PROD_CONN",
        "role": "ML_ENGINEER",
        "warehouse": "HEALTHCARE_ML_PROD_WH",
        ...
    },
}

def get_session() -> Session:
    session = Session.builder.config(
        "connection_name",
        CONFIG["connection_name"]
    ).create()
    ...
```

The Promotion Script

`scripts/promote_to_prod.sh`

```
Usage:  ./scripts/promote_to_prod.sh V2
        ./scripts/promote_to_prod.sh V3 --prod-repo /path/to/prod-repo
```

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              PROMOTION PIPELINE                              │
│                                                                              │
│  Step 1              Step 2            Step 3            Step 4       Step 5  │
│  ──────              ──────            ──────            ──────       ──────  │
│  Validate            Tag commit        (Optional)        Push tag     Snowflake│
│  working tree        with model        Copy to prod      to remote    Git FETCH│
│  - clean?            version           repo via rsync                        │
│  - on main?          model-V2                            git push     ALTER GIT│
│                      git tag -a                          origin       REPOSITORY│
│                                                          model-V2    ... FETCH │
└──────────────────────────────────────────────────────────────────────────────┘
```

Step	What Happens	Why
Validate	Checks for uncommitted changes, confirms `main` branch	Prevents promoting dirty code
Tag	Creates `model-V2` annotated tag	Version tracking, rollback point
Copy (opt.)	rsync `src/` + `production/` to separate prod repo	Isolate experiments from prod

Scheduled Execution with Snowflake Tasks

Task Chain

```
┌──────────────────┐
│ GIT_FETCH_TASK   │
│ Every 60 minutes │
│                  │
│ ALTER GIT REPO   │
│ ... FETCH        │
└──────────────────┘
        │
        │ AFTER
        v
┌──────────────────┐
│ BATCH_SCORING    │
│ _TASK            │
│                  │
│ EXECUTE IMMEDIATE │
│ FROM @.../main/  │
│ production/      │
│ run_batch_       │
│ inference.py     │
└──────────────────┘
```

> *The `AFTER` keyword chains tasks. Batch scoring only runs after a successful Git fetch.*

Task SQL

```sql
-- Task 1: Keep code in sync
CREATE OR REPLACE TASK
  HEALTHCARE_ML.TASKS.GIT_FETCH_TASK
  WAREHOUSE = HEALTHCARE_ML_WH
  SCHEDULE  = '60 MINUTE'
AS
  ALTER GIT REPOSITORY
    HEALTHCARE_ML.GIT_INTEGRATION
      .HEALTHCARE_ML_REPO FETCH;

-- Task 2: Score patients
CREATE OR REPLACE TASK
  HEALTHCARE_ML.TASKS.BATCH_SCORING_TASK
  WAREHOUSE = HEALTHCARE_ML_WH
  AFTER HEALTHCARE_ML.TASKS.GIT_FETCH_TASK
AS
  EXECUTE IMMEDIATE FROM
    @HEALTHCARE_ML.GIT_INTEGRATION
      .HEALTHCARE_ML_REPO
      /branches/main/production
      /run_batch_inference.py;

-- Enable (child first, then root)
ALTER TASK BATCH_SCORING_TASK RESUME;
ALTER TASK GIT_FETCH_TASK RESUME;
```

Monitoring Tasks

Check Task Run History

```snowflake
SELECT SCHEDULED_TIME, STATE, ERROR_MESSAGE, QUERY_ID
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    TASK_NAME => 'BATCH_SCORING_TASK',
    SCHEDULED_TIME_RANGE_START => DATEADD('HOUR', -24, CURRENT_TIMESTAMP()))
))
ORDER BY SCHEDULED_TIME DESC;
```

Optional: Weekly Retraining Task

```snowflake
CREATE OR REPLACE TASK HEALTHCARE_ML.TASKS.RETRAIN_TASK
    WAREHOUSE = HEALTHCARE_ML_WH
    SCHEDULE  = 'USING CRON 0 2 * * SUN America/Los_Angeles'   -- Sundays at 2am PT
AS
    EXECUTE IMMEDIATE FROM
      @HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO
      /branches/main/production/run_training.py;
```

> *Reads latest data from `RAW_DATA`, trains a new model, registers it in the Model Registry. Fully automated.*

Part 4: Team Workflow — Day in the Life

How a data science team collaborates using this setup

Team Workflow — Complete Cycle

```
DATA SCIENTIST                          GITHUB                          SNOWFLAKE
──────────────                          ──────                          ─────────

1. git clone repo
   ──────────────────>

2. git checkout -b
   feature/improve-bnp

3. Edit notebooks/
   Experiment locally
   Run: python -m src.train

4. Extract changes to src/

5. git add . && git commit
   git push origin feature/
   ──────────────────────────>
                                6. Open Pull Request
                                   Team reviews code
                                   + metrics diff

                                7. Merge to main
                                   ──────────────────────>
                                                            8. GIT_FETCH_TASK runs
                                                               (or manual FETCH)

                                                            9. BATCH_SCORING_TASK
                                                               runs new code auto-
                                                               matically

                                                            10. Results written to
                                                                BATCH_PREDICTIONS
```

Team Workflow — Step by Step

Step	Who	Action	Command / SQL
1	Data Scientist	Clone the repo	`git clone https://github.com/org/healthcare-readmission-ml.git`
2	Data Scientist	Create feature branch	`git checkout -b feature/improve-bnp-threshold`
3	Data Scientist	Experiment in notebooks	Open `notebooks/01_local_training.ipynb`, adjust features
4	Data Scientist	Extract to production code	Move logic to `src/feature_engineering.py`
5	Data Scientist	Test locally	`python -m src.train` then `python -m src.register_model --version V2_TEST`
6	Data Scientist	Push and open PR	`git push origin feature/improve-bnp-threshold`
7	Team Lead	Review and merge PR	GitHub PR review -- check code diff + metric changes
8	ML Engineer	Promote model	`./scripts/promote_to_prod.sh V2`
9	Snowflake	Auto-fetch code	`GIT_FETCH_TASK` runs every 60 minutes

Testing on Feature Branches

Run code from a feature branch directly in Snowflake

```sql
-- Fetch latest (includes all branches)
ALTER GIT REPOSITORY HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO FETCH;

-- List files on the feature branch
LIST @HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO
  /branches/feature/improve-bnp-threshold/production/;

-- Test the feature branch code WITHOUT affecting main
EXECUTE IMMEDIATE FROM
  @HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO
  /branches/feature/improve-bnp-threshold/production/run_training.py
  USING (VERSION => 'V2_TEST');
```

> *This lets you validate changes in Snowflake **before merging to main**. The scheduled Tasks still run from `main` -- your test is isolated.*

Snowflake Objects Summary

Everything Created in This Walkthrough

Schema	Object	Type	Purpose
`GIT_INTEGRATION`	`HEALTHCARE_ML_REPO`	Git Repository	Read-only clone of GitHub repo
`RAW_DATA`	`PATIENTS`, `ADMISSIONS`, `CLINICAL_MEASUREMENTS`	Tables	Source EHR data
`FEATURE_STORE`	`PATIENT`	Entity	Join key definition
`FEATURE_STORE`	`PATIENT_CLINICAL_FEATURES$V1`	Feature View (Dynamic Table)	33 auto-refreshed features
`MODEL_REGISTRY`	`READMISSION_PREDICTOR` V1	Model	GradientBoostingClassifier
`INFERENCE`	`BATCH_PREDICTIONS`	Table	Scored patient risk
`INFERENCE`	`BATCH_OUTPUT`	Stage	Parquet output from SPCS
`TASKS`	`GIT_FETCH_TASK`	Task	Hourly Git sync
`TASKS`	`BATCH_SCORING_TASK`	Task	Batch inference after fetch

Part 5: Hands-On Implementation

Follow these steps in your environment

Implementation Checklist

Phase 1: Foundation

- [ ] Create database: `CREATE DATABASE HEALTHCARE_ML;`
- [ ] Create schemas: `RAW_DATA`, `FEATURE_STORE`, `MODEL_REGISTRY`, `INFERENCE`, `GIT_INTEGRATION`, `TASKS`
- [ ] Create warehouse: `CREATE WAREHOUSE HEALTHCARE_ML_WH WAREHOUSE_SIZE = 'XSMALL';`
- [ ] Clone the Git repo locally

Phase 2: ML Pipeline (Notebooks 01-05)

- [ ] Run `01_local_training.ipynb` -- generate data + train model
- [ ] Run `02_snowflake_setup.ipynb` -- upload data, create Feature Store
- [ ] Run `03_model_registry.ipynb` -- register model
- [ ] Run `04_batch_inference.ipynb` -- batch scoring on SPCS
- [ ] Run `05_realtime_inference.ipynb` -- real-time prediction

Phase 3: Git Integration + Production

- [ ] Create API Integration (or reuse existing)

Common Issues & Solutions

Issue	Cause	Solution
`FETCH` fails with 403	Private repo, no secret	Create `GITHUB_SECRET` with a PAT and add `GIT_CREDENTIALS`
`API_ALLOWED_PREFIXES` error	Repo URL not in allowed list	Update `API INTEGRATION` to include your org URL
`EXECUTE IMMEDIATE` import error	`src/` not on Python path	Production entry points add project root to `sys.path`
Task not running	Tasks created but not resumed	`ALTER TASK ... RESUME` (child tasks first, then root)
Model version conflict	Version already exists	Use a new version name, or drop the existing version first
Features out of date	Dynamic Table lag	Check `SCHEDULING_STATE` in `DYNAMIC_TABLE_REFRESH_HISTORY()`

Key Takeaways

Snowflake Features Demonstrated

1. **Git Integration** -- connect repos, browse & execute code from SQL
2. **Feature Store** -- entities, feature views, Dynamic Tables, Online Store
3. **Model Registry** -- version, govern, and serve sklearn models
4. **SPCS Batch Inference** -- distributed scoring via Ray
5. **Tasks** -- scheduled, chained pipeline execution
6. **EXECUTE IMMEDIATE FROM** -- run `.py` files directly from Git stages

Operational Benefits

1. **Version control** -- every model version tied to a Git commit
2. **Reproducibility** -- same code in dev and prod, parameterized by env
3. **Audit trail** -- who changed what, when, reviewed by whom
4. **Automation** -- Git fetch + batch scoring on a schedule
5. **Isolation** -- test on feature branches without touching prod
6. **Least privilege** -- `ML_ENGINEER` role, not `ACCOUNTADMIN`

One Command to Promote

Thank You

Resources

Resource	Location
	`github.com/sfc-gh-moahmed/healthcare-readmission-ml`
Git setup SQL	`scripts/setup_snowflake_git.sql`
	`production/tasks/setup_tasks.sql`
Promotion script	`scripts/promote_to_prod.sh`